SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

MongoDB (CS62298CC)

# Assignment on

# "Vehicle Rental System"

Submitted by

| | |
|---|---|
| **Likitha R** | **4SF23CS093** |
| **Nethra Vaikunt Prabhu** | **4SF23CS127** |
| **Sanjana M** | **4SF23CS193** |
| **Sharanya R** | **4SF23CS201** |

In partial fulfillment of the requirements for the VI Semester

BACHELOR OF ENGINEERING
In

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of
**Mr.Harisha / Mrs. Aparna Krishnan**
Assistant Professor, Dept. of CS&E

2025-26

| QR Code and URL | Max Marks | Marks Obtained |
|---|---|---|
| https://vehicle-rental-system-drive-flow.vercel.app/ | 20 | |

# ABSTRACT

DriveFlow is *Vehicle Rental System* is a comprehensive full-stack web application designed to streamline vehicle rental operations through efficient database management and user-friendly interfaces. This project leverages MongoDB as the primary NoSQL database to store and manage vehicle inventory, user accounts, booking transactions and payment records.

It is developed using Next.js and MongoDB Atlas that aims to simplify and digitize the process of renting vehicles. The system provides two distinct portals: an *Admin Portal* and a *User Portal*, each designed to handle specific functionalities efficiently.

The Admin Portal enables administrators to manage vehicle listings by adding, updating, and deleting vehicle details such as name, brand, fuel type, seating capacity, pricing, location, and images. It also facilitates booking management by allowing admins to track and update booking statuses (confirmed, cancelled, completed) and monitor payment records. The User Portal allows users to browse available vehicles, apply filters based on location, price, and availability, and make bookings for a selected date range. Users can also choose a payment method (UPI, card, or cash) via a simulated payment system and manage their bookings, including cancellations.

The application uses MongoDB Atlas, a cloud-based NoSQL database service, for efficient data storage and management. MongoDB stores data in flexible, JSON-like documents, making it highly suitable for handling dynamic and scalable applications. Collections such as vehicles, bookings, users, and payments are structured to support real-time operations and ensure data consistency.

Overall, the system demonstrates a practical implementation of a vehicle rental platform with key features such as inventory management, booking validation, and payment tracking, providing a user-friendly and scalable solution.

# ACKNOWLEDGEMENT

The success and final outcome of this Project report preparation required a lot of guidance and assistance from many people and we are extremely fortunate to have got this all along the completion of our Mini Project. Any task would not be successful without sincere efforts from various people. It gives great pleasure to extent our thanks and gratitude to those who have been instrumental in completion of this project.

We are profoundly indebted to our guide, **Mr.Harisha and Mrs.Aparna Krishnan** , Assistant Professor, Department of Computer Science & Engineering for their innumerable acts of timely advice, encouragement and we sincerely express our gratitude.

We are grateful to **Dr. Mustafa Basthikodi**, Head of the department of CSE, for providing us with right atmosphere in the department, encouraging and supporting us, which made our task appreciable.

We are indebted to our principal **Dr. S. S. Injaganeri** and the **Management** of Sahyadri College of Engineering & Management, for having provided all facilities for the completion of the project.

We would also like to express our sincere gratitude to our **Parents, all teaching and non-teaching staff, lab assistants, and our friends** from the Department of Computer Science & Engineering, who have directly or indirectly contributed to the successful completion of this Project Report.

**Likitha R 4SF23CS093**
**Nethra Vaikunt Prabhu 4SF23CS127**
**Sanjana M 4SF23CS193**
**Sharanya R 4SF23CS201**

# Contents

# Chapter 1

## INTRODUCTION

The rapid advancement of web technologies has significantly transformed the vehicle rental industry, enabling businesses to shift from traditional, manual processes to web-based platforms to enhance customer experience and operational efficiency [1]. Modern vehicle rental systems are expected to provide seamless user experiences while managing complex operations such as vehicle inventory tracking, booking management, payment processing, user profiles and real-time availability updates. These growing requirements demand robust, scalable, and flexible system architectures capable of handling diverse and dynamic datasets.

Traditional relational databases have been used to support such systems; however, they often impose rigid schema constraints that can limit flexibility in handling dynamic and diverse data. To address these limitations, this project adopts MongoDB, a NoSQL database which provides a document-oriented approach that allows for flexible schema design and better scalability. This is advantageous in vehicle rental systems, where different vehicles may have varying attributes.

The Vehicle Rental System developed in this project is built using Next.js, a modern full-stack framework that supports both frontend and backend development within a unified environment. The application is divided into two primary modules: the *Admin Portal* and the *User Portal*. The Admin Portal enables administrators to manage vehicle listings, monitor bookings, and track payments, while the User Portal allows customers to browse available vehicles, apply filters, make bookings based on date ranges, and perform cancellations.

To facilitate interaction with MongoDB, the system utilizes Mongoose, which provides schema-based modeling and validation for application data. Overall, the system serves as a practical implementation of a scalable and user-friendly vehicle rental platform, aligning with current industry trends in web application development.

# Chapter 2

# PROBLEM FORMULATION

## 2.1 Problem Statement

Existing vehicle rental management systems often struggle with inefficient data organization, limited scalability, and difficulty in adapting to changing business requirements. Traditional SQL-based systems require rigid schema definitions that complicate modifications, and normalized structures can result in complex join operations affecting query performance. This project addresses the need for a flexible, scalable database solution that can efficiently handle diverse vehicle attributes, complex booking relationships, and real-time availability tracking while maintaining data consistency and security.

## 2.2 Problem Description

Vehicle rental systems must handle multiple interconnected operations, including maintaining vehicle inventories, managing booking schedules, and processing payments. In traditional or poorly designed systems, challenges can arise. Additionally, the absence of a structured yet adaptable data management approach makes it difficult to scale the system or accommodate changing requirements.

The primary challenges in developing a vehicle rental system include:

- Managing diverse vehicle types with varying specifications and attributes
- Efficient querying of vehicle availability across multiple time periods
- Secure user authentication and authorization with role-based access control
- Rapid iteration and schema modifications without system downtime
- Scalability to handle increasing booking volumes and double bookings

## 2.3 Objectives of Project

The primary objectives of this technical seminar are outlined as follows:

1. To design and implement MongoDB collections for **Users**, **Vehicles**, **Bookings**, and **Payments** with appropriate schema validation and referencing strategies.

2. To design a system capable of managing vehicle listings, bookings, and payments in an organized manner.

3. To ensure efficient indexing strategies to optimize query performance for common operations such as vehicle search, availability checks, and user bookings.

4. To ensure accurate tracking of vehicle availability and prevent booking conflicts.

5. To provide a simple and user-friendly interface for both administrators and users.

# Chapter 3

# METHODOLOGY AND IMPLEMENTATION

## 3.1 Overview

This chapter outlines the methodology and implementation of the Vehicle Rental System developed using Next.js and MongoDB Atlas. It explains the design approach, system structure, development process, and practical considerations for building and deploying the application.

## 3.2 Methodology

A modular and iterative development approach is followed, where the system is divided into smaller components such as vehicle management, booking, and payments. Each module is designed, developed, and tested independently before integration.

### 3.2.1 Theoretical Frameworks

The system is based on the following key frameworks:

- Client-server architecture for handling frontend and backend communication
- NoSQL data modeling using document-based storage
- CRUD operations for managing system data
- Validation logic for ensuring booking constraints and data consistency

### 3.2.2 System Components

- User Interface (Admin and User portals)
- Backend APIs for data operations
- Database for storing vehicles, bookings, and payments
- Booking validation module

## 3.3 System Architecture

The system follows a three-tier architecture:

1. **Presentation Layer**: Built with Next.js for user interaction
2. **Application Layer**: API routes handling business logic

3. **Data Layer**: MongoDB Atlas storing application data

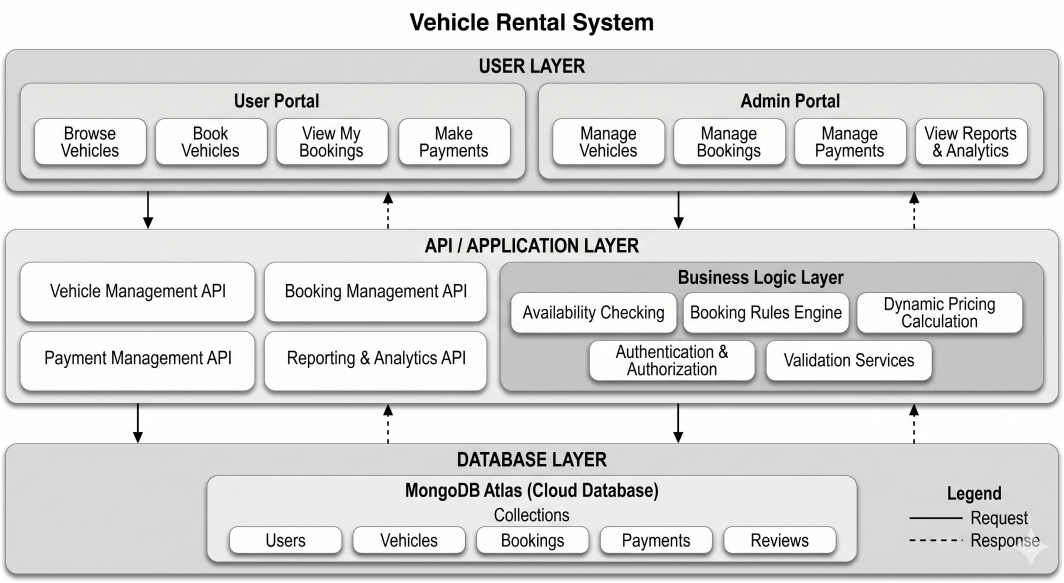


Figure 3.1: System Architecture of Vehicle Rental System

## 3.4 Setup Requirements

### 3.4.1 Hardware Requirements

- Computer with minimum 4GB RAM (8GB recommended)
- Stable internet connection

### 3.4.2 Software Requirements

- Node.js and npm
- Next.js
- MongoDB Atlas account
- Code editor (e.g., Visual Studio Code)
- Web browser

## 3.5 Implementation Steps

### 3.5.1 Backend Development

1. Design database schemas for vehicles, bookings, and payments
2. Create API routes for CRUD operations

3. Implement booking validation to prevent overlaps

### 3.5.2 Frontend Development

1. Develop landing page and portals
2. Create forms for vehicle management and booking
3. Implement filters and user interaction features

### 3.5.3 Testing

- Unit testing for API routes
- Functional testing for booking and payment flows
- Validation testing for edge cases like double booking

## 3.6 Configuration Parameters

- MongoDB connection URI
- Environment variables for deployment
- API endpoints configuration
- Date and pricing calculation settings

## 3.7 Challenges and Solutions

- Preventing double booking: Implemented date overlap validation
- Handling dynamic data: Used flexible schema design in MongoDB
- State management: Ensured smooth data flow between frontend and backend

## 3.8 Replication Steps

1. Clone the project repository.
2. Install dependencies using `npm install`.
3. Configure environment variables (MongoDB URI).
4. Run the development server.
5. Access the system through a web browser or mobile browser.

# Chapter 4

# PROJECT RESULT

## 4.1 Seminar Outcome

### 4.1.1 Knowledge Gained

The Vehicle Rental System project provided a comprehensive understanding of NoSQL databases, with a specific focus on MongoDB. Key concepts such as document-oriented data modeling, schema design, collections, documents, indexing, aggregation pipelines, and integration with Node.js using Mongoose were explored in depth.

### 4.1.2 Exposure to Technical Literature

The project enhanced the ability to interpret and analyze technical literature related to MongoDB. This included official documentation, best practice guidelines, and research studies on NoSQL database performance, scalability, and real-world applications.

### 4.1.3 Technical Writing Skills

Significant improvement was achieved in technical documentation skills, particularly in describing database schemas, data relationships, and system design decisions. The process also involved presenting the justification for selecting MongoDB over traditional relational databases in a structured and academically appropriate manner.

### 4.1.4 Publication and Documentation Experience

Experience was gained in preparing structured technical reports highlighting database architecture, implementation details, and system performance considerations. Special attention was given to documenting MongoDB configurations, schema design choices, and optimization techniques in a formal and publishable format.

### 4.1.5 Communication Skills

The ability to communicate complex database concepts such as flexible schema design, aggregation operations, and NoSQL advantages was enhanced. The use of diagrams and structured explanations improved clarity in presenting how MongoDB supports the Vehicle Rental System to both technical and non-technical audiences.

### 4.1.6 Technical Skill Development

The project helped enhance practical skills in technologies such as Next.js and MongoDB, along with experience in API development and database design.

### 4.1.7 Awareness of Sustainable Development Goals (SDGs)

The project increased awareness of the role of efficient data management in supporting Sustainable Development Goals, particularly SDG 9 (Industry, Innovation, and Infrastructure) and SDG 11 (Sustainable Cities and Communities).

### 4.1.8 Contribution Towards SDGs

The future enhancements in Vehicle Rental System such as real-time analytics and intelligent fleet optimization can further contribute to sustainable urban mobility solutions aligned with SDG 9 and SDG 11.

## Chapter 5

# CONCLUSION

The Vehicle Rental System developed in this project demonstrates an effective approach to building a modern, full-stack web application that addresses key challenges in managing vehicle rentals. By integrating features such as vehicle management, booking validation, and payment tracking, the system provides a structured and user-friendly platform for both administrators and users.

The use of Next.js enables seamless development of both frontend and backend components within a unified framework, while MongoDB Atlas offers a flexible and scalable solution for handling dynamic and diverse data. The implementation successfully ensures important constraints such as preventing double bookings, enforcing minimum rental duration, and maintaining data consistency.

Overall, the project highlights the practical application of NoSQL databases and modern web technologies in solving real-world problems. It also provides valuable insights into system design, data modeling, and application deployment. The developed system can be further enhanced with advanced features such as authentication, real-time availability updates, and integration with payment gateways, making it suitable for real-world deployment and scalability.

# REFERENCES


[1] P. Sharma and R. Kumar, "Digital Transformation in Vehicle Rental Industry: A Comprehensive Study," *International Journal of Technology Management*, vol. 42, no. 3, pp. 234–251, 2024.

[2] J. Pokorny, "NoSQL Databases: A Survey and Future Perspectives," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 1, pp. 89–104, 2024.

[3] MongoDB Inc., "MongoDB Schema Design Best Practices," 2024. [Online]. Available: `https://docs.mongodb.com/manual/core/data-model-design/`

[4] S. Verma and A. Singh, "Query Optimization in Document-Based Databases," *Journal of Database Systems*, vol. 28, no. 2, pp. 145–162, 2023.

[5] S. Newman, *Building Microservices with MongoDB*. O'Reilly Media, 2023.

[6] M. Tiwari and N. Gupta, "Performance Comparison: SQL vs NoSQL Databases in Web Applications," in *International Conference on Cloud Computing*, pp. 456–471, 2024.

[7] K. Chodorow, *MongoDB: The Definitive Guide*, 4th ed. O'Reilly Media, 2023.

[8] D. Jacobson et al., "RESTful API Design: Best Practices and Patterns," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1–35, 2024.

## Appendix A

# APPENDICES

## Course Outcome

- CO 1:Identify the research papers/applied knowledge resources on latest trends in area of interest and formulate objectives of the study.
- CO 2:Acquaint literatures review methods and identify the significant technical information relevant to selected topic.
- CO 3:Interpret the observations with hypothesis and summarize the conclusions.
- CO 4:Make use of a logical thought process to efficiently sift findings and produce a well-structured, tailored report.
- CO 5:Develop a detailed presentation of observational outcomes and recommendations for improving future scope.